Web Applications

JavaScript Promises and async/await

Suhel Hammoud

Promises and async/await in JavaScript (ES2015+)

In this lecture, you'll learn how JavaScript handles asynchronous operations using **Promises** and the **async/await** syntax introduced in ES2017.

You will also learn how to write cleaner, more maintainable asynchronous code.

Why Asynchronous Programming?

- JavaScript is **single-threaded**.
- Long-running operations can block execution.
- Examples include:
  - Network requests
  - Reading files
  - Timers
  - Database queries
- Asynchronous programming allows JavaScript to continue executing other tasks while waiting for these operations to complete.

Asynchronous Programming

Example

```
const data = fetch('https://jsonplaceholder.typicode.com/posts');

console.log('This runs immediately!');
```

What happens?

- `fetch()` starts the HTTP request
- It immediately returns a **Promise**
- JavaScript continues running the next line
- The response becomes available later

✅ Non-blocking execution improves performance and responsiveness.

▓▓ What is a Promise?

A **Promise** is an object that represents the eventual completion or failure of an asynchronous operation.

A Promise has three possible states:

- 🕒 **Pending** → operation still running
- ✅ **Fulfilled** → operation completed successfully
- ❌ **Rejected** → operation failed

Promises allow asynchronous code to be handled in a structured and predictable way.

Creating and Using Promises

```
function fetchData() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const rnd = Math.random();
            if (rnd < .5) resolve("Data received!")
            else reject("Failed to fetch data");
        }, 2000);
    })
}
fetchData().then(data => console.log(data))
           .catch(err => console.error(err));
```

Important Methods

- `.then()` handles successful results
- `.catch()` handles errors or rejections
- `.finally()` runs regardless of success or failure

▓▓ Chaining Promises

Promises can be chained together for sequential asynchronous operations.

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

▒▒ Key Idea

- First `.then()` returns a new Promise
- Values can be passed to the next step
- Errors automatically propagate to `.catch()`

✅ This avoids deeply nested callbacks.

▓▓ Common Problem: Callback Hell

Before Promises, asynchronous code often became deeply nested.

```
setTimeout(() => {
  console.log('Step 1');
  setTimeout(() => {
    console.log('Step 2');
    setTimeout(() => {
      console.log('Step 3');
    }, 1000);
  }, 1000);
}, 1000);
```

😵 Problems with callback hell:

- Difficult to read
- Hard to debug
- Poor maintainability
- Complex error handling

▓▓ Solving Callback Hell with Promises

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

delay(1000)
  .then(() => {
    console.log('Step 1');
    return delay(1000);
  })
  .then(() => {
    console.log('Step 2');
    return delay(1000);
  })
  .then(() => {
    console.log('Step 3');
  });
```

✅ Promises create cleaner sequential flows.

Solving Callback Hell with async/await

```
const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

async function runSteps() {
  await delay(1000);
  console.log('Step 1');

  await delay(1000);
  console.log('Step 2');

  await delay(1000);
  console.log('Step 3');
}
runSteps();
```

Advantages

- Easier to read
- Looks like synchronous code
- Cleaner logic and structure

🚀 async/await is now the preferred style in modern JavaScript.

What is async/await?

`async/await` was introduced in **ES2017 (ES8)**.

It is built on top of Promises and provides cleaner syntax for asynchronous code.

```
async function loadPosts() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts');
  const data = await response.json();
  console.log(data);
}
```

Benefits

- Improved readability
- Easier debugging
- Better error handling
- Cleaner sequential logic

▓▓ The async Keyword

The async keyword:

- Declares an asynchronous function
- Automatically returns a Promise
- Allows the use of await inside the function

```
async function getUser() {
  return 'John Doe';
}

getUser().then(name => {
  console.log(name);
});  // Output: John Doe
```

✅ Even regular return values become resolved Promises.

▓▓ The await Keyword

The `await` keyword:

- Pauses execution inside an async function
- Waits until a Promise settles
- Returns the resolved value

```
async function fetchData() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts');
  const data = await response.json();
  console.log(data);
}
```

⚠️ Important:

- `await` only works inside `async` functions
- It does NOT block the JavaScript event loop
- Only the current async function pauses

▓▓ Error Handling with async/await

Use try/catch blocks to handle asynchronous errors.

```
async function loadData() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/invalid-url');
    const data = await response.json();
    console.log(data);
  } catch (error) { console.error('Error:', error)}
}
```

✅ try/catch makes async error handling cleaner and easier to understand.

Parallel Execution with Promise.all()

Sometimes multiple asynchronous tasks can run simultaneously.

```
async function loadParallel() {
  const [posts, users] = await Promise.all([
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(r => r.json()),

    fetch('https://jsonplaceholder.typicode.com/users')
      .then(r => r.json())
  ]);
  console.log(posts);
  console.log(users);
}
```

Benefits

- Executes tasks concurrently
- Faster than sequential execution
- Improves application performance

Sequential vs Parallel Execution

Sequential Execution

```
const a = await fetch('/api/a');
const b = await fetch('/api/b');
```

⏳ Total waiting time = A + B

Parallel Execution

```
const [a, b] = await Promise.all([
  fetch('/api/a'),
  fetch('/api/b')
]);
```

⚡ Both requests execute at the same time.

When to Use Promises vs async/await

Situation	Recommended Approach
Simple chaining	.then().catch()
Sequential logic	async/await
Complex workflows	async/await
Parallel execution	Promise.all()
Clean error handling	try/catch

General Recommendation

✅ Prefer async/await for readability and maintainability.

▓▓ Summary

- JavaScript uses asynchronous programming to avoid blocking execution.
- A Promise represents a future value or error.
- Promises can be:
  - Pending
  - Fulfilled
  - Rejected
- `async` functions always return Promises.
- `await` pauses execution until a Promise settles.
- `try/catch` handles asynchronous errors cleanly.
- `Promise.all()` runs tasks in parallel for better performance.

🚀 Modern JavaScript applications heavily rely on Promises and async/await.